ЛЕКЦИЯ

Работа приложения, API, сервер и инфраструктура

Путь одного действия через браузер, обработку и хранение

Преподаватель



Основная литература

Муромцев В. В. Информационные технологии в профессиональной деятельности : учебник и практикум / В. В. Муромцев, А. В. Муромцева. — Москва, Вологда : Инфра-Инженерия, 2023. — 383 с. — ISBN 978-5-9729-1299-5.

Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт].

Открыть источник ↗

Мейер Б. Основы объектно-ориентированного проектирования : учебник для СПО / Б. Мейер. — 2-е изд. — Саратов : Профобразование, 2024. — 751 с. — ISBN 978-5-4488-1002-2.

Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт].

Открыть источник ↗

Дополнительная литература

Крынецкая Г. С. Вычислительные машины, сети и системы : учебник / Г. С. Крынецкая. — Москва : Издательский Дом МИСиС, 2023. — 614 с. — ISBN 978-5-907560-73-4.

Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт].

Открыть источник

Алексахин А. Н. Программное обеспечение управления проектами : учебник / А. Н. Алексахин, И. Ю. Владыко, Е. В. Сибирская ; под редакцией Е. В. Сибирской. — Москва : Университет «Синергия», 2025. — 144 с. — ISBN 978-5-4257-0670-6.

Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт].

Открыть источник

Материалы к занятиям

Ссылка на материалы

https://disk.yandex.ru/d/P40mjNJWYJAW9g

Отсканируйте QR-код или откройте ссылку.

ВОПРОСЫ ТЕМЫ

Вопросы темы

01 Части приложения

02 HTTP и JSON

03 Договор API

04 Обработка на сервере

05 Работа с базой

06 Путь заявки № 104

07 Размещение приложения

08 Выпуск версии

ВОПРОС 1

Части приложения

Показать состояние и передать пользовательские данные

Ответственность сервера

Сеанс — сведения о ранее подтверждённом входе; сервер связывает с ними каждый запрос.

Ответственность браузера

Подготовка
Получить выбранный проект и мотивацию

Отправка
Передать согласованное обращение

Представление
Показать ожидание, результат или неопределённость

Ответственность сервера

Установить пользователя
Получить Анну из действующего сеанса

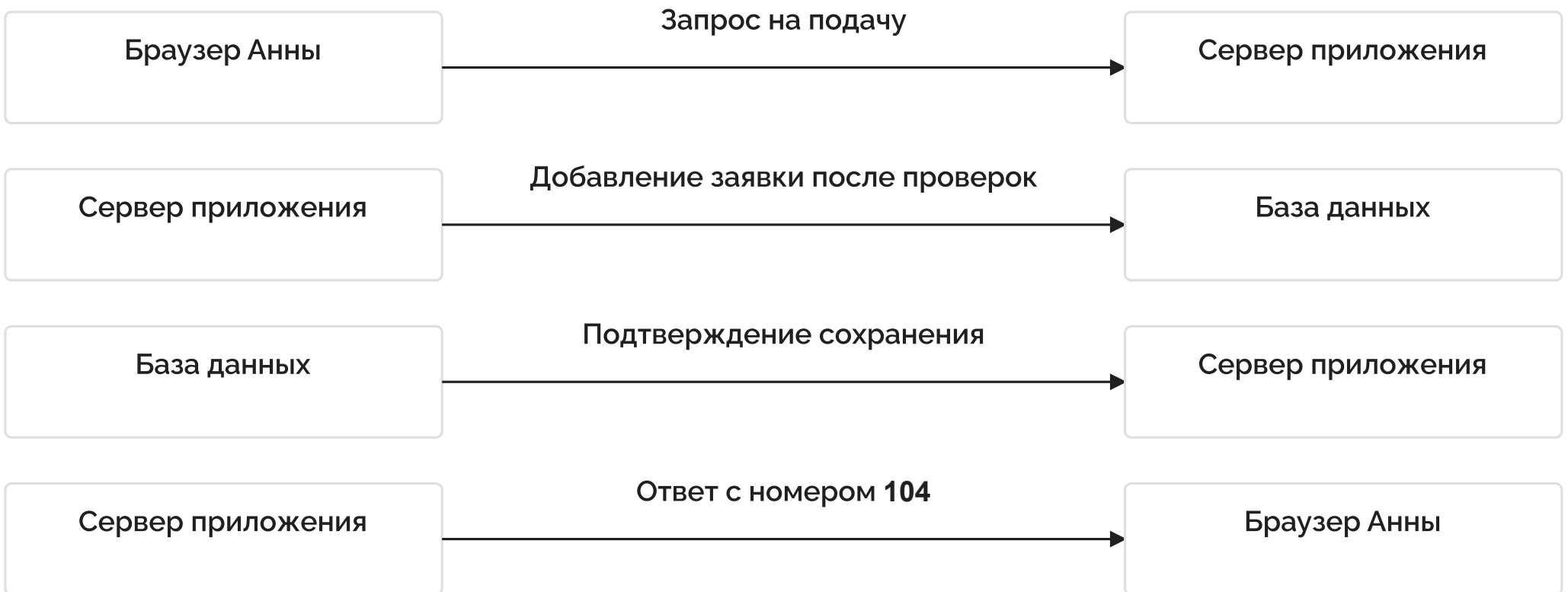
Проверить
Формат, роль, проект, текст и повтор

Сохранить
Создать согласованную запись

Ответить
Сообщить подтверждённый результат

Роль базы в обмене

Сохранённое состояние переживает завершение запроса



Практика: Части приложения

ЗАДАЧА

Учебный случай: клиент бронирования отправляет user_id=77.

ЧТО ПРОВЕРИТЬ

Выполните три шага: получите проверяемый результат, сверяйте его с условиями и исправляйте расхождения. После каждого шага меняйтесь ролями.

Командный разбор · 3 шага · проверка каждого результата

https://olgakraven.github.io/2026-PIS-lecture/?webinar=3&slide=9&mode=activity

Части приложения

Учебный случай: клиент бронирования отправляет user_id=77. Сеанс принадлежит пользователю 52. JSON можно изменить вручную. Операция выполняется от текущего пользователя.

Аналитик выделяет правило · Оператор записывает результат · Проверяющий ищет контрпример. Меняйтесь ролями после каждого шага.

1. Установите источник личности

В JSON user_id=77, а серверный сеанс принадлежит 52. Аналитик записывает доверенный ID и значение из недоверенного тела.

Доверенный ID: _____

ID из тела: _____

Правило и проверка: _____

2. Соберите серверную запись

Для бронирования выбран учебный сеанс 23, поле session_id. Это ID встречи, а не сеанс входа. Оператор создаёт пару (user_id, session_id), подставляя личность по сеансу.

user_id: _____

session_id: _____

Правило и проверка: _____

3. Проверьте подмену

Проверяющий повторил запрос с user_id=88 в теле при том же сеансе. Укажите владельца записи и число записей, которые запрос вправе создать от имени 88.

Владелец записи: _____

Записей от имени 88: _____

Правило и проверка: _____

Прогноз → проверенные результаты → уточнённый вывод. Сверяйте каждый шаг в интерактивном дополнении; обоснование обсудите с преподавателем.

Самопроверка · Вопрос 1

Браузер отправил id пользователя в JSON. Можно доверять этому id?

Ваш ответ

- ○ **А.** Да, это JSON
- ○ **Б.** Нет, сервер устанавливает личность по сеансу
- ○ **В.** Да, если поле скрыто

ВОПРОС 2

HTTP и JSON

Правила запроса и ответа между участниками веб-обмена

Адрес запроса

HTTP передаёт запрос и ответ; URL задаёт адрес ресурса, а метод — действие над ним.

Запишите

HTTP — протокол обмена сообщениями запроса и ответа между клиентом и сервером.

HTTP

```
GET /api/projects
Accept: application/json
```

Упрощённая запись запроса каталога. Версия протокола и технические заголовки опущены.

Адрес запроса

Часть адреса	Учебный пример	Смысл
Схема	https	Защищённое соединение
Имя узла	projects.example.test	Условный адрес приложения
Путь	/api/projects/17	Проект 17
Полный пример	https://projects.example.test/api/projects/17	Не является действующим сервисом

Метод запроса

GET, POST и PATCH выражают разные действия

Метод	В нашем договоре	Пример
GET	Получить сведения	/api/projects
POST	Создать заявку	/api/applications
PATCH	Частично изменить заявку	/api/applications/104

Заголовки

Заголовки описывают сообщение: Content-Type — его тело, Accept — желаемый формат ответа.

```
Content-Type: application/json
Accept: application/json
```

Content-Type описывает передаваемое содержимое. Accept задаёт предпочтительный формат ответа.

Тело запроса

Передаются выбранный проект и мотивация

```
{
  "project_id": 17,
  "motivation": "Хочу разработать каталог мероприятий и личный кабинет участника студенческого клуба."
}
```

Тело создания заявки. student_id и status клиент не задаёт.

Ответ сервера

Код и содержимое описывают результат операции

```
201 Created
Location: /api/applications/104
Content-Type: application/json

{"id":104,"project_id":17,
 "student_id":12,"status":"pending"}
```

Сокращённый пример успешного ответа после сохранения.

Формат JSON

JSON — текстовый формат: объект содержит пары «имя: значение», массив — упорядоченный список.

Фрагмент	Тип	Смысл
"project_id": 17	Число	Идентификатор выбранного проекта
"motivation": "..."	Строка	Текст из формы
"is_open": true	Логическое значение	Признак открытого набора

Группы результатов

Успех, отказ по запросу и сбой обработки различаются

Код	Смысл в договоре
200 / 201	Запрос выполнен / ресурс создан
401 / 403	Не подтверждён вход / недостаточно прав
404	Запрашиваемый ресурс не найден
409	Конфликт с текущим состоянием
422 / 5xx	Недопустимое содержимое / сбой на стороне сервера

Практика: Запрос и ответ

ЗАДАЧА

Учебный случай: POST /registrations с event_id=42 и комментарием.

ЧТО ПРОВЕРИТЬ

Выполните три шага: получите проверяемый результат, сверяйте его с условиями и исправляйте расхождения. После каждого шага меняйтесь ролями.

Командный разбор · 3 шага · проверка каждого результата

https://olgakraven.github.io/2026-PIS-lecture/?webinar=3&slide=19&mode=activity

Запрос и ответ

Учебный случай: POST /registrations с event_id=42 и комментарием. Это запрос на создание. Тело имеет корректный синтаксис JSON. Доступность события ещё не проверена.

Аналитик выделяет правило · Оператор записывает результат · Проверяющий ищет контрпример. Меняйтесь ролями после каждого шага.

1. Разберите запрос

Учебный контракт: POST /registrations; тело JSON {event_id:42, comment:"Хочу участвовать"}. Для этого упражнения длина комментария допустима. Аналитик извлекает метод и ID объекта.

Метод: _____

event_id: _____

Правило и проверка: _____

2. Составьте серверный конвейер

A — записать и зафиксировать; B — проверить поля и длину; C — разобрать JSON; D — проверить сеанс, доступность события и уникальность. Оператор задаёт порядок.

Порядок: _____

Первый шаг, проверяющий доступность: _____

Правило и проверка: _____

3. Проверьте предметный отказ

Событие 42 оказалось закрыто. Проверяющий фиксирует число новых строк и последний выполненный шаг из A–D.

Новых строк: _____

Последний шаг: _____

Правило и проверка: _____

Прогноз → проверенные результаты → уточнённый вывод. Сверяйте каждый шаг в интерактивном дополнении; обоснование обсудите с преподавателем.

Самопроверка · Вопрос 2

JSON прочитан без синтаксической ошибки, но мотивация содержит 49 символов. Что следует?

Ваш ответ

- ○ **А.** Формат корректен, но предметное правило длины не выполнено
- ○ **Б.** Любой корректный JSON гарантирует 201
- ○ **В.** Content-Type автоматически делает длину допустимой

ВОПРОС 3

Договор API

Операция включает путь, данные, права и результат

Получение каталога

GET /api/projects возвращает проекты

Запишите

API — согласованный интерфейс взаимодействия программных компонентов.

API как договор

Обращение
Метод, путь и допустимое содержимое

Условия
Кто вызывает и какие правила выполнены

Результат
Успешное представление или согласованная ошибка

Получение каталога

```
GET /api/projects

200 OK
[{"id":17,"title":"Сайт студенческого клуба",
  "teacher_id":7,"is_open":true}]
```

Сокращённое представление каталога, без полного описания каждого проекта.

Получение проекта

GET /api/projects/17 возвращает выбранное описание

```
GET /api/projects/17

200 OK
{"id":17,"title":"Сайт студенческого клуба",
 "description":"Каталог мероприятий и личный кабинет",
 "teacher_id":7,"is_open":true}
```

Данные для карточки выбранного проекта.

Создание заявки

POST /api/applications принимает только проект и мотивацию

```
POST /api/applications
Content-Type: application/json

{
  "project_id": 17,
  "motivation": "Хочу разработать каталог мероприятий и личный кабинет участника студенческого клуба."
}
```

Упрощённый запрос. Действующий сеанс подразумевается, технические заголовки опущены.

Получение своих заявок

GET /api/my/applications использует текущего пользователя

```
GET /api/my/applications

200 OK
[{"id":104,"project_id":17,
  "student_id":12,"status":"pending"}]
```

Сокращённый ответ для Анны. Отбор по текущему пользователю выполняет сервер.

Получение заявок проекта

GET /api/projects/17/applications требует права на проект

```
GET /api/projects/17/applications

200 OK
[{"id":104,"project_id":17,"student_id":12,
  "status":"pending"}]
```

Сокращённый ответ Ирине, чей id 7 совпадает с teacher_id проекта.

Решение по заявке

PATCH /api/applications/104 меняет только допустимый статус

1

Пользователь

Установить Ирину по сеансу; не доверять id из тела запроса.

2

Принадлежность

Найти проект заявки; teacher_id должен совпасть с пользователем.

3

Текущее состояние

UPDATE applications SET status=$2 WHERE id=$1 AND status='pending' RETURNING id;

4

Результат записи

Одна изменённая строка — решение; ноль — перечитать состояние и вернуть согласованный отказ.

$1 — номер заявки, $2 — новое решение. После проверки прав меняют только pending.

Свой проект; status=pending; новый status=accepted.

1
Пользователь

Установить Ирину по сеансу; не доверять id из тела запроса.

2
Принадлежность

Найти проект заявки; teacher_id должен совпасть с пользователем.

3
Текущее состояние

UPDATE applications SET status=$2 WHERE id=$1 AND status='pending' RETURNING id;

4
Результат записи

Одна изменённая строка — решение; ноль — перечитать состояние и вернуть согласованный отказ.

Личность и принадлежность подтверждены.

Условное UPDATE меняет только pending.

Изменена одна строка; после COMMIT можно подтвердить результат.

200 OK · accepted

$1 — номер заявки, $2 — новое решение. После проверки прав меняют только pending.

Свой проект; status=accepted; попытка перевести в rejected.

1
Пользователь

Установить Ирину по сеансу; не доверять id из тела запроса.

2
Принадлежность

Найти проект заявки; teacher_id должен совпасть с пользователем.

3
Текущее состояние

UPDATE applications SET status=$2 WHERE id=$1 AND status='pending' RETURNING id;

4
Результат записи

Одна изменённая строка — решение; ноль — перечитать состояние и вернуть согласованный отказ.

Условие status=pending не совпало.

Изменено 0 строк; проверенный актуальный статус уже итоговый.

409 INVALID_TRANSITION · accepted сохранён

$1 — номер заявки, $2 — новое решение. После проверки прав меняют только pending.

Единый формат ошибок

Машинный код причины и текст выполняют разные задачи

```
409 Conflict
Content-Type: application/json

{"code":"APPLICATION_EXISTS",
 "message":"Вы уже подали заявку на этот проект."}
```

Тело ошибки содержит устойчивый **code** и понятное пользователю **message**.

Практика: Договор API

ЗАДАЧА

Учебный случай: endpoint моих регистраций вызван пользователем 52.

ЧТО ПРОВЕРИТЬ

Выполните три шага: получите проверяемый результат, сверяйте его с условиями и исправляйте расхождения. После каждого шага меняйтесь ролями.

Командный разбор · 3 шага · проверка каждого результата

https://olgakraven.github.io/2026-PIS-lecture/?webinar=3&slide=29&mode=activity

Договор API

Учебный случай: endpoint моих регистраций вызван пользователем 52. В базе есть регистрации пользователей 52 и 77. Сеанс подтверждён. Параметр user_id в URL может быть подменён.

Аналитик выделяет правило · Оператор записывает результат · Проверяющий ищет контрпример. Меняйтесь ролями после каждого шага.

1. Сформулируйте фильтр

Учебный GET /my-registrations выполняется в сеансе 52. Аналитик заполняет поле фильтра владельца и значение.

Поле: _____

Значение: _____

Правило и проверка: _____

2. Вычислите результат

Таблица: 730(user_id=52), 731(77), 732(52). Оператор перечисляет ID строк ответа через пробел; порядок не важен.

ID ответа: _____

Количество строк: _____

Правило и проверка: _____

3. Проверьте подмену параметра

Проверяющий добавил ?user_id=77 к маршруту my-registrations. Контракт игнорирует этот параметр и сохраняет фильтр сеанса. Укажите ID ответа и число чужих строк.

ID ответа: _____

Чужих строк: _____

Правило и проверка: _____

Прогноз → проверенные результаты → уточнённый вывод. Сверяйте каждый шаг в интерактивном дополнении; обоснование обсудите с преподавателем.

Самопроверка · Вопрос 3

GET своих заявок возвращает чужие записи. Что нарушено?

Ваш ответ

- ○ **А.** Фильтрация по установленному пользователю
- ○ **Б.** Только отбор по проекту без проверки владельца
- ○ **В.** Только фильтрация списка в браузере

ВОПРОС 4

Обработка на сервере

Упорядоченный алгоритм выполняет одну операцию

Определение пользователя

Сервер проверяет сведения о сеансе; student_id из тела не является доказательством личности.

Серверный обработчик

```
пользователь = определить_по_сеансу()
проверить_вход_и_роль(пользователь)
данные = разобрать_и_проверить_тело()
проверить_проект_и_мотивацию(данные)
заявка = сохранить_без_повтора(пользователь, данные)
вернуть_подтверждение(заявка)
```

Псевдокод создания заявки: обозначает смысл шагов, а не синтаксис языка.

Определение пользователя

Источник	Можно считать доказательством личности?	Почему
Доверенный контекст сеанса	Да, в принятом учебном допущении	Вход уже проверен системой
Поле student_id в JSON	Нет	Клиент может изменить значение
Имя на картинке формы	Нет	Внешний вид не устанавливает пользователя

Проверка входных данных

Договор длины: trim убирает пробелы с концов, затем считаем кодовые точки Unicode.

Проверка	Корректный вариант	Отказ
Структура	Объект с двумя ожидаемыми полями	Неверное содержимое
project_id	Числовой идентификатор проекта	Отсутствует или неподходящий тип
motivation	Текст после согласованной нормализации	Нет текста или нарушена длина
Лишние поля	Отсутствуют	Не принимаются как полномочие

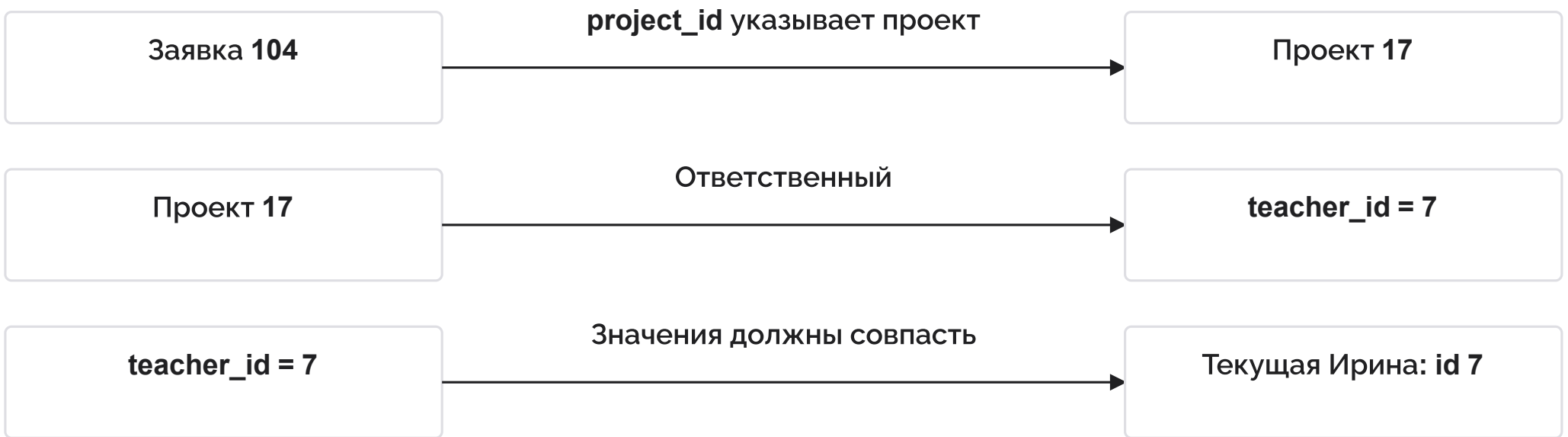
Проверка роли

Установленный пользователь получает только разрешённые действия

Операция	Требуемая роль	Пример отказа
Создание заявки	student	Ирина не подаёт заявку как преподаватель
Решение по заявке	teacher	Анна не принимает собственную заявку
Список проекта	teacher и принадлежность	Роль без связи проекта недостаточна

Принадлежность проекта

Ирина принимает решение по связи teacher_id



Бизнес-правила

Бизнес-правило — ограничение предметной области: открытый набор, допустимая длина, одна пара и разрешённый переход.

01

Набор открыт

Проект существует и принимает новые заявки

02

Мотивация допустима

Нормализованный текст имеет 50–1000 символов

03

Повтора нет

Сочетание студента и проекта ещё не занято

04

Переход допустим

Решение применяется только из pending

Обработка отказов

Неправильный путь завершается без недопустимой записи

Причина	Код ответа	Машинная причина
Не подтверждён вход	401	AUTH_REQUIRED
Недостаточно прав	403	FORBIDDEN
Повтор	409	APPLICATION_EXISTS
Проект закрыт	409	PROJECT_CLOSED
Длина мотивации	422	MOTIVATION_LENGTH

Формирование успешного ответа

Подтверждение опирается на результат сохранения

01

Сохранение завершилось

Получен номер созданной заявки

02

Собраны допустимые поля

id, project_id, student_id и status

03

Отправлен ответ

201 Created с сокращённым представлением

Практика: Проверки сервера

ЗАДАЧА

Учебный случай: организатор 11 меняет событие владельца 12.

ЧТО ПРОВЕРИТЬ

Выполните три шага: получите проверяемый результат, сверяйте его с условиями и исправляйте расхождения. После каждого шага меняйтесь ролями.

Командный разбор · 3 шага · проверка каждого результата

https://olgakraven.github.io/2026-PIS-lecture/?webinar=3&slide=39&mode=activity

Проверки сервера

Учебный случай: организатор 11 меняет событие владельца 12. Роль организатора подтверждена. event.owner_id=12. Специального общего доступа нет.

Аналитик выделяет правило · Оператор записывает результат · Проверяющий ищет контрпример. Меняйтесь ролями после каждого шага.

1. Восстановите проверку доступа

Исполнитель 11 — организатор; owner_id объекта 12; специальных полномочий нет. Аналитик извлекает оба ID.

Исполнитель: _____

Владелец: _____

Правило и проверка: _____

2. Остановите мутацию

Оператор выполняет проверки: A — установить сеанс; B — обновить; C — сравнить owner_id; D — загрузить объект. Укажите последний шаг и число обновлений.

Последний шаг: _____

Обновлений: _____

Правило и проверка: _____

3. Проверьте положительный контроль

Проверяющий использует объект owner_id=11 с прежними остальными допустимыми условиями. Укажите owner_id, проходящий проверку, и число обновлений одной успешной команды.

Проходящий owner_id: _____

Обновлений: _____

Правило и проверка: _____

Прогноз → проверенные результаты → уточнённый вывод. Сверяйте каждый шаг в интерактивном дополнении; обоснование обсудите с преподавателем.

Самопроверка · Вопрос 4

Ирина имеет роль teacher и владеет проектом, но заявка уже accepted. Что проверить?

Ваш ответ

- ○ **А.** Только наличие роли: этого достаточно
- ○ **Б.** Допустимость перехода из текущего состояния
- ○ **В.** Только синтаксис JSON, состояние не относится к правам

ВОПРОС 5

Работа с базой

Запрос выбирает нужные строки и поля

Добавление записи

SQL — язык работы с таблицами; $1, $2, $3 обозначают значения, переданные отдельно от текста запроса.

Чтение данных

```
SELECT id, title, is_open
FROM projects
WHERE id = $1;
```

Учебный SQL. Параметр $1 получает 17 отдельно от текста запроса.

Добавление записи

```
INSERT INTO applications
  (project_id, student_id, motivation, status, created_at)
VALUES ($1, $2, $3, 'pending', CURRENT_TIMESTAMP)
RETURNING id;
```

Параметры: проект 17, установленный студент 12 и допустимая мотивация. id назначает база.

Связанные данные в SQL

Соединение даёт название проекта для своей заявки

```
SELECT a.id, p.title, a.status
FROM applications AS a
JOIN projects AS p ON p.id = a.project_id
WHERE a.student_id = $1;
```

Параметр $1 берётся из текущего пользователя: для Анны это 12.

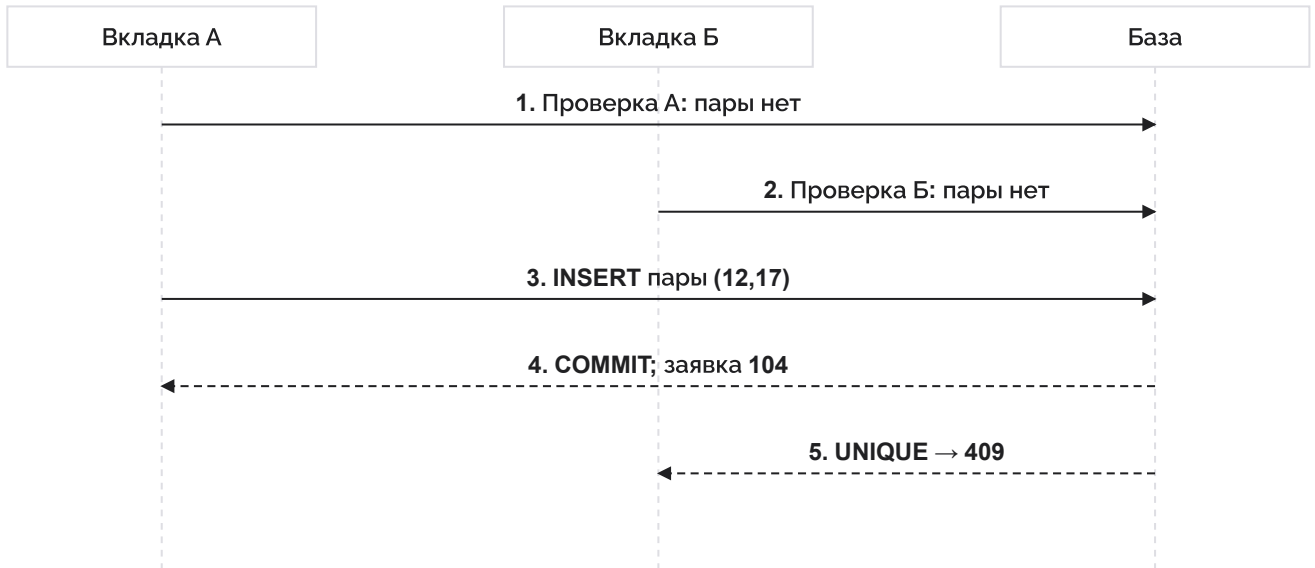
Ограничения базы

Разные ограничения защищают разные свойства данных

Ограничение	Что предотвращает	Чего не доказывает
PRIMARY KEY	Повтор ключа строки	Право пользователя
FOREIGN KEY	Ссылку на отсутствующую запись	Нужную роль пользователя
NOT NULL	Отсутствие обязательного значения	Допустимую длину текста
UNIQUE пары	Повтор студента и проекта	Открытость набора
CHECK статуса	Неизвестное значение	Разрешённый переход

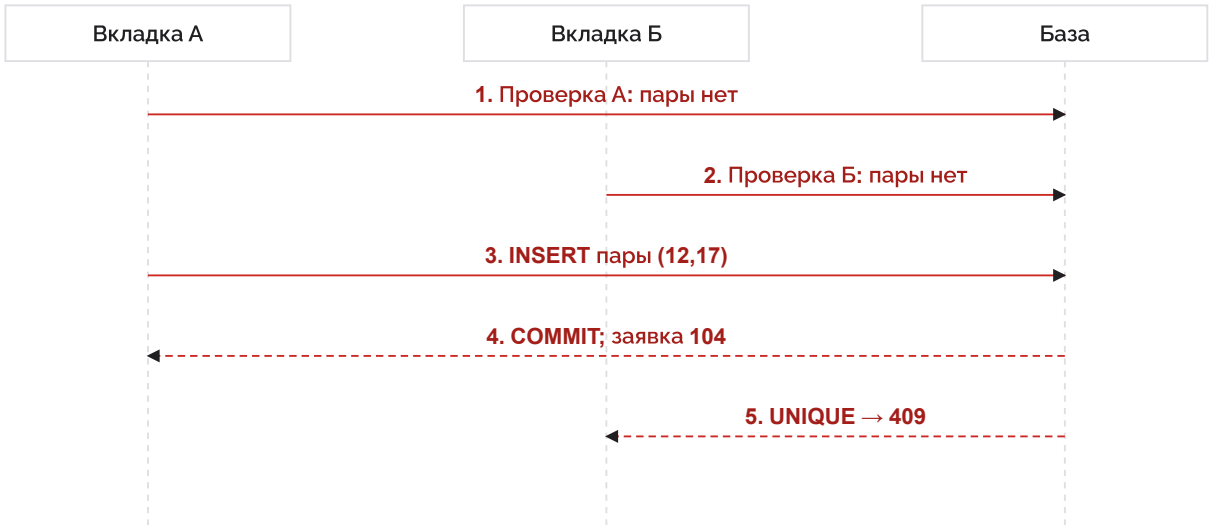
Два одновременных запроса

Оба могут прочитать отсутствие заявки до первой записи



BEGIN — начало; INSERT — добавление; COMMIT — фиксация. UNIQUE не пропускает вторую одинаковую пару.

Обе вкладки: student_id=12, project_id=17. Обе успели прочитать отсутствие заявки.



Обе предварительные проверки: пары ещё нет.

Вкладка А: INSERT занимает уникальную пару.

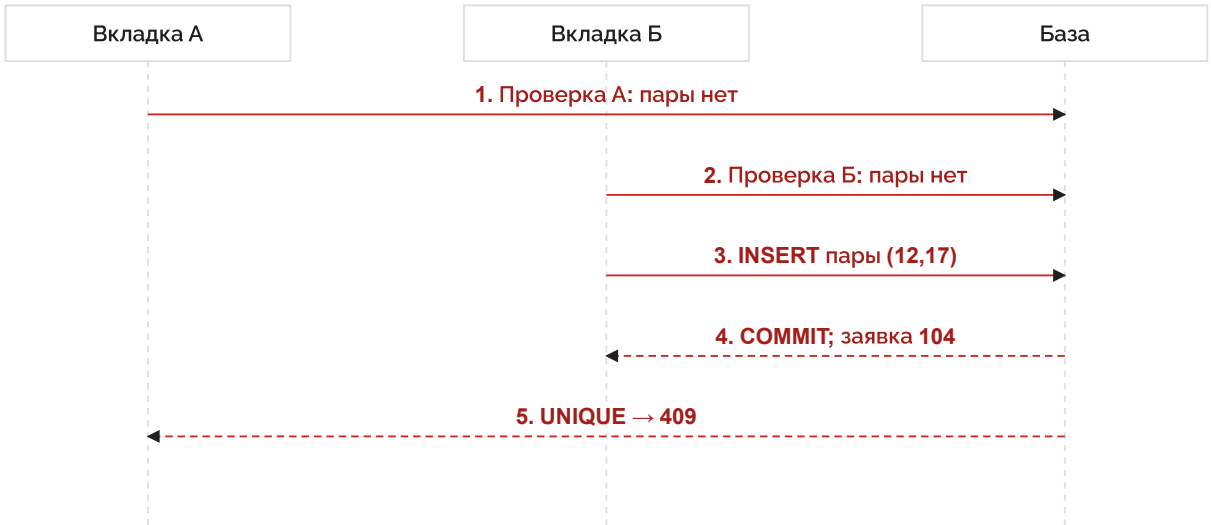
Вкладка А: COMMIT подтверждён.

Другая вкладка: UNIQUE отклоняет повтор той же пары.

Одна запись; победитель 201; другая попытка 409 APPLICATION_EXISTS

BEGIN — начало; INSERT — добавление; COMMIT — фиксация. UNIQUE не пропускает вторую одинаковую пару.

Обе вкладки: student_id=12, project_id=17. Обе успели прочитать отсутствие заявки.



Обе предварительные проверки: пары ещё нет.

Вкладка Б: INSERT занимает уникальную пару.

Вкладка Б: COMMIT подтверждён.

Другая вкладка: UNIQUE отклоняет повтор той же пары.

Одна запись; победитель 201; другая попытка 409 APPLICATION_EXISTS

BEGIN — начало; INSERT — добавление; COMMIT — фиксация. UNIQUE не пропускает вторую одинаковую пару.

Как приложение обрабатывает конфликт UNIQUE

Нарушение уникальности в базе переводят в понятную ошибку API, не создавая вторую запись.

01

База

UNIQUE отклоняет занятую пару (12,17)

02

Обработчик

Отменяет неуспешную операцию и распознаёт конкретное ограничение

03

Ответ

409 APPLICATION_EXISTS; клиент предлагает проверить свою заявку

Транзакция

BEGIN начинает транзакцию, COMMIT фиксирует её, ROLLBACK отменяет незавершённые изменения.

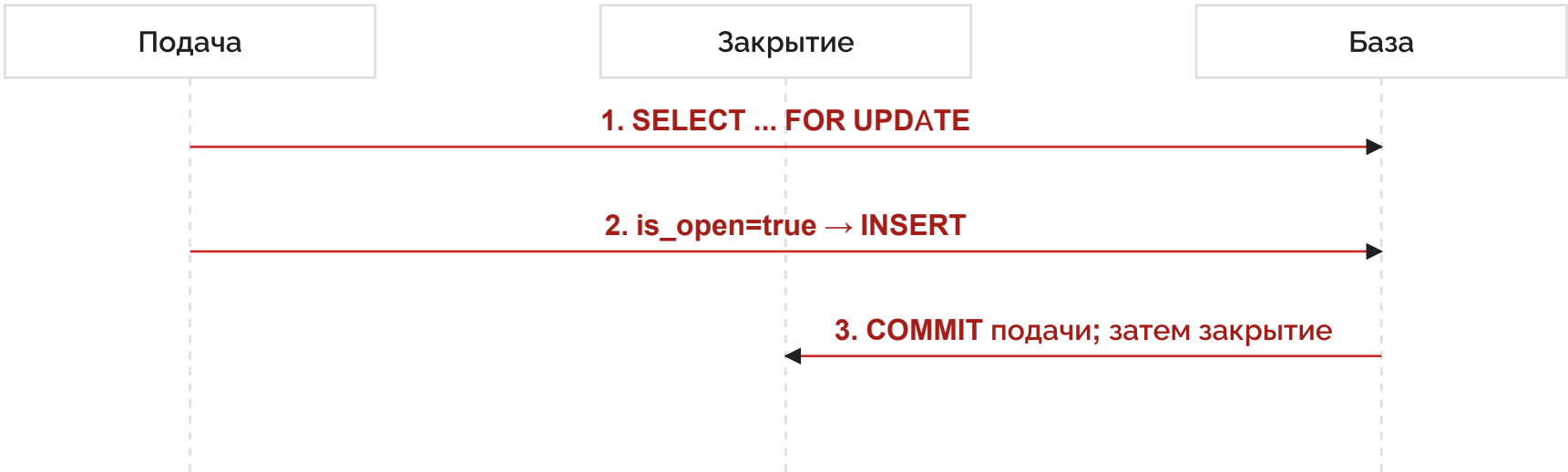
Запишите

Транзакция — единица работы с данными, изменения которой фиксируются вместе либо отменяются.



FOR UPDATE блокирует строку проекта до конца транзакции. Подача и закрытие используют ту же блокировку.

Подача первой блокирует projects.id=17; проект ещё открыт.



Подача: SELECT ... FOR UPDATE блокирует строку проекта.

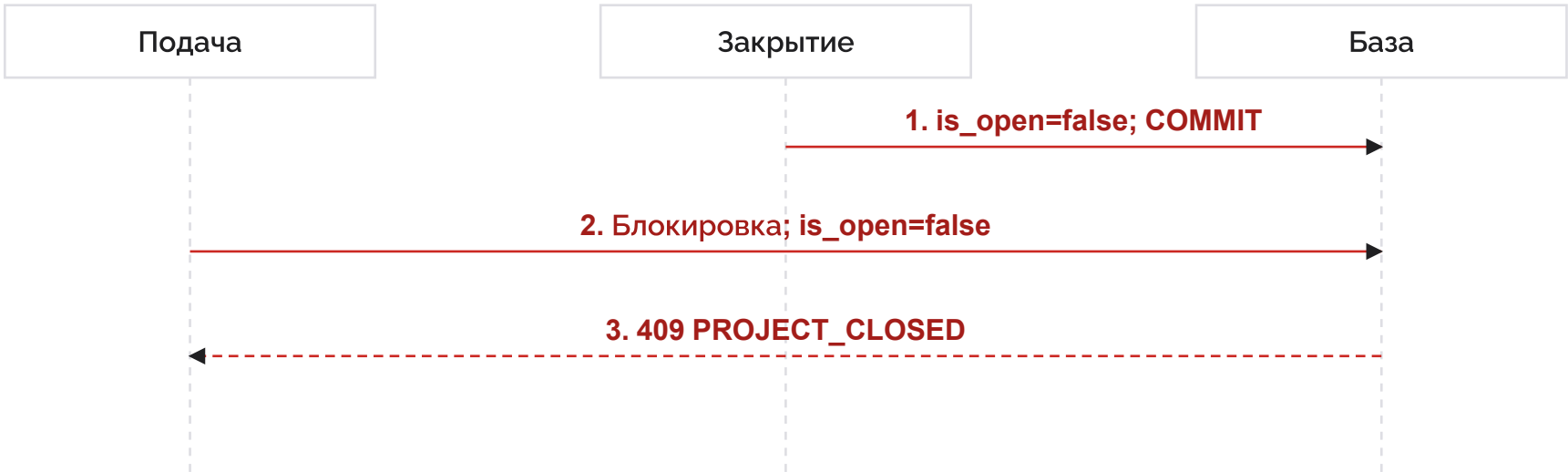
Подача проверяет is_open=true и вставляет заявку.

COMMIT подачи; затем закрытие получает блокировку и меняет is_open=false.

Заявка сохранена до закрытия; будущие подачи запрещены.

FOR UPDATE блокирует строку проекта до конца транзакции. Подача и закрытие используют ту же блокировку.

Закрытие первым блокирует ту же строку и фиксирует is_open=false.



Закрытие: UPDATE projects SET is_open=false; COMMIT.

Подача получает блокировку и читает уже закрытый проект.

Условие is_open ложно: INSERT не выполняется.

409 PROJECT_CLOSED · новой заявки нет

FOR UPDATE блокирует строку проекта до конца транзакции. Подача и закрытие используют ту же блокировку.

Сохранение перед подтверждением

Номер от INSERT ещё не подтверждает сохранение; успех клиенту отправляют после COMMIT.

01

Изменение подготовлено

Запись ещё относится к незавершённой работе

02

Изменение зафиксировано

Система получила подтверждение завершения

03

Ответ отправлен

Клиент может показать успешную подачу

Практика: Конкуренция

ЗАДАЧА

Учебный случай: два клиента одновременно бронируют ту же пару (52,42).

ЧТО ПРОВЕРИТЬ

Выполните три шага: получите проверяемый результат, сверяйте его с условиями и исправляйте расхождения. После каждого шага меняйтесь ролями.

Командный разбор · 3 шага · проверка каждого результата

https://olgakraven.github.io/2026-PIS-lecture/?webinar=3&slide=49&mode=activity

Конкуренция

Учебный случай: два клиента одновременно бронируют ту же пару (52,42). Оба прочитали «регистрации нет». Ограничение UNIQUE(user_id,event_id) активно. Обе транзакции пытаются вставить пару.

Аналитик выделяет правило · Оператор записывает результат · Проверяющий ищет контрпример. Меняйтесь ролями после каждого шага.

1. Проследите гонку

Два клиента A и B одновременно создают пару (52,42); оба SELECT ещё вернули пусто. Аналитик записывает число прочитавших отсутствие и поля UNIQUE.

Прочитавших отсутствие: _____

Поля ограничения: _____

Правило и проверка: _____

2. Выполните расписание

Порядок учебного прогона: A INSERT+COMMIT, затем B INSERT той же пары. Оператор указывает успешных фиксаций и конфликтов при UNIQUE.

Успешных фиксаций: _____

Конфликтов: _____

Правило и проверка: _____

3. Проверьте инвариант

Проверяющий читает таблицу после обеих попыток. Сколько строк пары (52,42) должно быть? Сколько успешных подтверждений создания вправе показать сервер?

Строк пары: _____

Подтверждений создания: _____

Правило и проверка: _____

Прогноз → проверенные результаты → уточнённый вывод. Сверяйте каждый шаг в интерактивном дополнении; обоснование обсудите с преподавателем.

Самопроверка · Вопрос 5

Два запроса одновременно проверили отсутствие пары. Что защищает от двух записей?

Ваш ответ

- ○ **А.** Только предварительный SELECT
- ○ **Б.** Уникальное ограничение в базе и обработка конфликта
- ○ **В.** Только блокировка кнопки

ВОПРОС 6

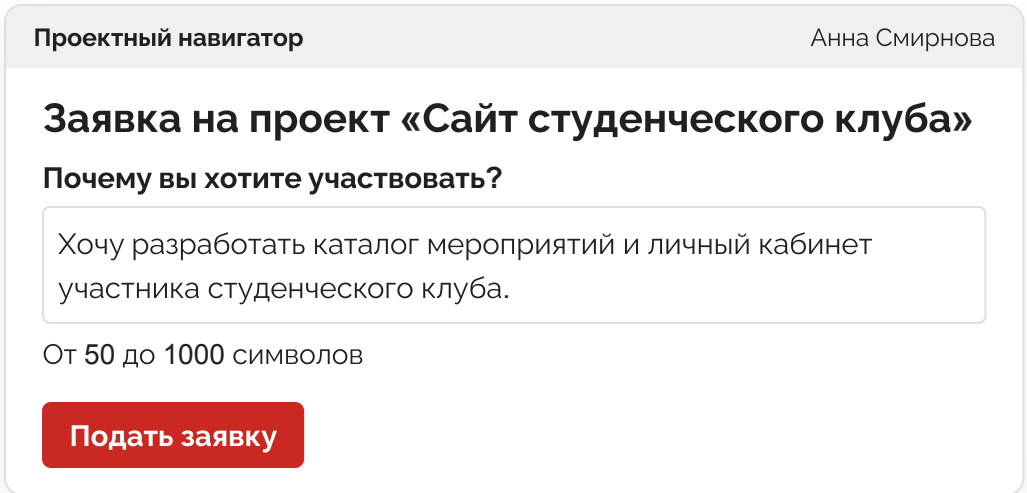
Путь заявки № 104

Перед отправкой выполнены пользовательские условия

Полный путь: от формы к запросу

Связываем уже разобранные уровни: проект 17 из формы станет project_id, а Анну установит сеанс.

Путь заявки: исходная форма



Путь заявки: запрос

```
POST /api/applications
Content-Type: application/json

{
  "project_id": 17,
  "motivation": "Хочу разработать каталог мероприятий и личный кабинет участника студенческого клуба."
}
```

Шаг 2 полного пути. Сеанс Анны подразумевается и не заменяется полем student_id.

Путь заявки: установленный пользователь

Авторство заявки определяет проверенный сеанс Анны 12, а не поле JSON или подпись макета.

Проверяемое сведение	Источник	Результат шага
Пользователь	Действующий сеанс	id 12
Роль	Сохранённые сведения пользователя	student
Свободный номер из тела	В договоре отсутствует	Не используется как личность

Путь заявки: проверка проекта

Проект существует и открыт для новой подачи

Сведение проекта 17	Учебное значение	Вывод
Наличие записи	Проект найден	Можно продолжить предметные проверки
is_open	true	Новый приём разрешён
teacher_id	7	Будущая заявка попадёт в область Ирины

Полный путь: одинаковый подсчёт длины

После trim: «A😀» содержит 2 кодовые точки, но JavaScript length даёт 3; договор использует точки.

```
const normalized = motivation.trim();
const n = [...normalized].length;
const valid = n >= 50 && n <= 1000;
```

Правило длины одинаково в форме и на сервере; пробелы внутри текста сохраняются.

Путь заявки: сохранение

Пара 12 и 17 создаётся один раз

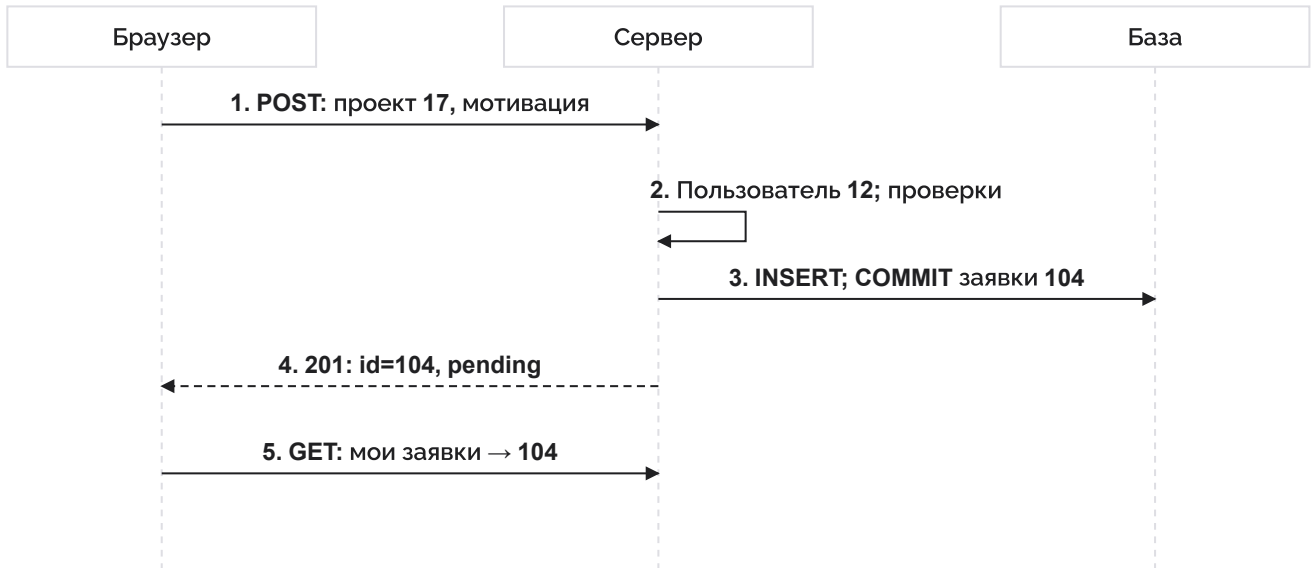


Схема сообщений; порядок сверху вниз, без временной шкалы.

Анна 12; проект 17 открыт; мотивация допустима; пары ещё нет.



POST /api/applications: project_id=17 и motivation.

Сервер устанавливает пользователя 12 и проверяет условия.

INSERT создаёт pending; COMMIT подтверждает сохранение.

Ответ 201: id=104, status=pending.

GET /api/my/applications подтверждает доступную Анне запись.

Одна заявка 104; сохранение предшествует подтверждению.

Схема сообщений; порядок сверху вниз, без временной шкалы.

Полный путь: связать ответ с записью

201 подтверждает создание; Location указывает адрес новой заявки, id связывает ответ с записью.

```
201 Created
Location: /api/applications/104
Content-Type: application/json

{"id":104,"project_id":17,
 "student_id":12,"status":"pending"}
```

Шаг 7. Сокращённый ответ после фиксации записи.

Путь заявки: обновление интерфейса

Подтверждение отражает полученные серверные сведения

Проектный навигатор Анна Смирнова

Заявка сохранена

Браузер получил 201, показал заявку 104, проект 17 и состояние «На рассмотрении».

Заявка сохранена

Заявка № 104 · Сайт студенческого клуба

На рассмотрении

Полный путь: проверка после нового входа

После нового входа сервер снова устанавливает Анну 12 и возвращает ей сохранённую заявку 104.

```
GET /api/my/applications

200 OK
[{"id":104,"project_id":17,
  "student_id":12,"status":"pending"}]
```

Сокращённый результат повторного чтения Анной.

Полный путь: другой пользователь той же записи

Ирина 7 читает заявку 104 через свой проект 17; две роли видят одну запись с разными полномочиями.

```
GET /api/projects/17/applications

200 OK
[{"id":104,"project_id":17,
  "student_id":12,"status":"pending"}]
```

Ирина установлена как id 7; teacher_id проекта 17 равен 7.

Практика: Фиксация и ответ

ЗАДАЧА

Учебный случай: запись регистрации 730 создана в транзакции.

ЧТО ПРОВЕРИТЬ

Выполните три шага: получите проверяемый результат, сверяйте его с условиями и исправляйте расхождения. После каждого шага меняйтесь ролями.

Командный разбор · 3 шага · проверка каждого результата

https://olgakraven.github.io/2026-PIS-lecture/?webinar=3&slide=61&mode=activity

Фиксация и ответ

Учебный случай: запись регистрации 730 создана в транзакции. INSERT прошёл. COMMIT завершился ошибкой. Успешный ответ клиенту ещё не отправлен.

Аналитик выделяет правило · Оператор записывает результат · Проверяющий ищет контрпример. Меняйтесь ролями после каждого шага.

1. Разделите события транзакции

INSERT вернул ID 730, COMMIT сообщил ошибку; исход фиксации ещё не установлен. Аналитик записывает известный ID и число подтверждённых фиксаций.

Известный ID: _____

Подтверждённых фиксаций: _____

Правило и проверка: _____

2. Уточните исход чтением

Для упражнения чтение GET /registrations/730 доступно текущему владельцу. Оператор записывает метод и путь проверки без новой вставки.

Метод: _____

Путь: _____

Правило и проверка: _____

3. Разберите наблюдаемую ветвь

В данном прогоне повторное чтение достоверно вернуло запись 730. Проверяющий указывает число найденных записей и число дополнительных POST, нужных для её создания.

Найдено: _____

Дополнительных POST: _____

Правило и проверка: _____

Прогноз → проверенные результаты → уточнённый вывод. Сверяйте каждый шаг в интерактивном дополнении; обоснование обсудите с преподавателем.

Самопроверка · Вопрос 6

После восстановления связи GET возвращает заявку 104, а прежний POST завершился таймаутом. Что установлено?

Ваш ответ

- ○ **А.** Заявка 104 существует и доступна текущему пользователю
- ○ **Б.** Прежний POST точно не выполнялся
- ○ **В.** Нужно создать заявку повторно

ВОПРОС 7

Размещение приложения

Одинаковая программа может иметь разные настройки и данные

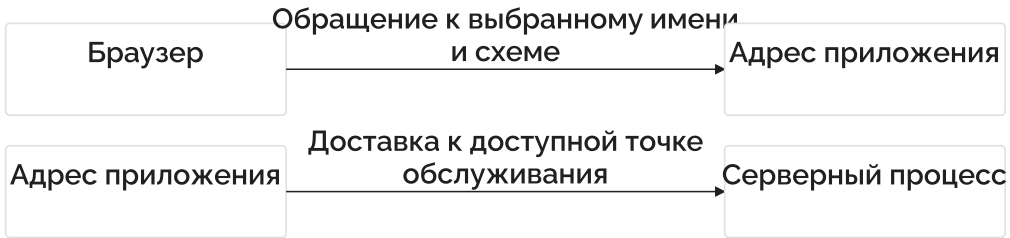
Адрес приложения

Браузеру нужен доступный сетевой узел и путь

Локальная и рабочая среда

Среда	Назначение	Данные
Локальная	Разработка и подготовленные проверки	Учебный набор
Рабочая	Использование ресурса по общему адресу	Рабочие записи
Условие перехода	Настройки согласованы отдельно	Данные не подменяются автоматически

Адрес приложения



Доменное имя

DNS (система доменных имён) сопоставляет имя узла с сетевыми адресами для подключения.

01

Имя

Условный projects.example.test читается человеком

02

Разрешение имени

Система доменных имён помогает определить адрес узла

03

Обращение

Браузер соединяется с точкой обслуживания

HTTPS

HTTPS — HTTP поверх TLS: защищает обмен и позволяет проверить сервер по сертификату.

Средство	Что обеспечивает	Чего не заменяет
TLS в HTTPS	Защиту обмена и проверку стороны по принятой модели сертификатов	Правильность бизнес-правил
Серверная авторизация	Право Анны или Ирины на действие	Защиту сетевой передачи
Ограничения базы	Согласованность записей	Проверку сетевого сертификата

Процесс сервера

Программа должна оставаться запущенной и доступной

01

Запуск

Рабочая версия начинает принимать обращения

02

Работа

Процесс выполняет обработчики и использует настройки

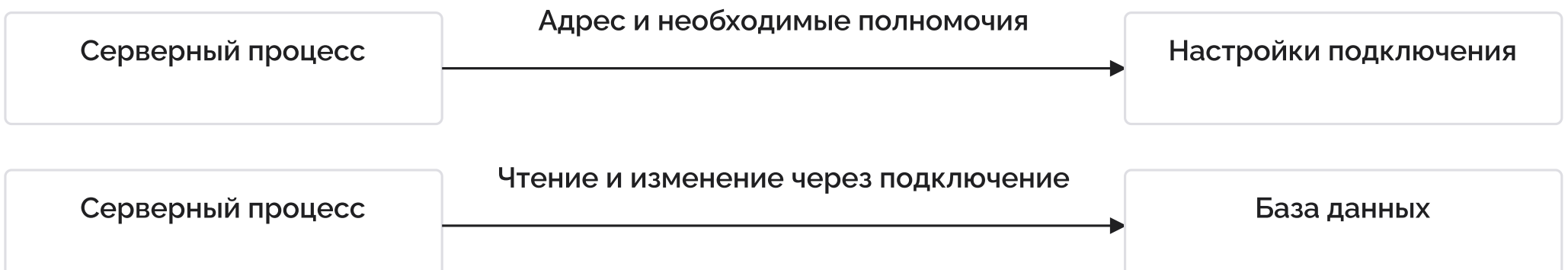
03

Остановка

Новые обращения не получают обычную обработку

Подключение к базе

Обработка использует отдельные параметры доступа к данным



Конфигурация

Параметры среды отделяют от правил и пользовательских данных

Параметр	Учебное обозначение	Назначение
Адрес базы	DB_URL = значение среды	Подключение к нужному хранилищу
Режим	APP_ENV = production	Условия запуска
Адрес приложения	APP_ORIGIN = рабочий адрес	Согласованное обращение
Доступ	Секреты не включены в пример	Не публикуются в клиентском коде

Доступность компонентов

Рабочая страница не доказывает исправность полного пути

Компонент	Признак	Ограничение вывода
Страница интерфейса	Загрузилась оболочка	API может быть недоступен
API каталога	Получен ожидаемый список	Запись может не работать
База	Доступны нужные операции	Права пользователя ещё проверяются
Полный сценарий	Получен согласованный результат	Нужны также отрицательные проверки

Практика: Конфигурация

ЗАДАЧА

Учебный случай: сервис перенесён на другой сервер.

ЧТО ПРОВЕРИТЬ

Выполните три шага: получите проверяемый результат, сверяйте его с условиями и исправляйте расхождения. После каждого шага меняйтесь ролями.

Командный разбор · 3 шага · проверка каждого результата

https://olgakraven.github.io/2026-PIS-lecture/?webinar=3&slide=71&mode=activity

Конфигурация

Учебный случай: сервис перенесён на другой сервер. Новая база имеет другой адрес. Браузер работает через API. Подключение к базе выполняет сервер.

Аналитик выделяет правило · Оператор записывает результат · Проверяющий ищет контрпример. Меняйтесь ролями после каждого шага.

1. Составьте карту подключений

Браузер обращается к API https://app.test/api, сервер — к DB_HOST=db-new. Аналитик называет компонент, которому нужна настройка базы, и компонент, который вызывает API.

Кому нужен DB_HOST: _____

Кто вызывает API: _____

Правило и проверка: _____

2. Измените конфигурацию

Учебные переменные: DB_HOST, API_URL, LOG_LEVEL. После переноса базы меняется только её адрес. Оператор указывает имя изменяемой переменной и значение.

Переменная: _____

Значение: _____

Правило и проверка: _____

3. Проверьте маршрут после запуска

A — браузер→API; B — API→база; C — браузер→база. Проверяющий выбирает необходимые проверки связи и число секретов базы, допустимых в клиентском JS.

Проверяемые связи: _____

Секретов базы в JS: _____

Правило и проверка: _____

Прогноз → проверенные результаты → уточнённый вывод. Сверяйте каждый шаг в интерактивном дополнении; обоснование обсудите с преподавателем.

Самопроверка · Вопрос 7

Куда передавать пароль базы при выпуске?

Ваш ответ

- ○ **А.** В клиентский JavaScript
- ○ **Б.** В защищённую конфигурацию сервера
- ○ **В.** В публичную ссылку

ВОПРОС 8

Выпуск версии

Изменение проходит подготовку, размещение и подтверждение

Подготовка настроек выпуска

Рабочая версия получает согласованную конфигурацию

Выпуск версии

Выбор версии
Известно, какой проверенный вариант выпускают

Подготовка
Настройки, совместимость данных и резервирование

Размещение
Обновление и запуск по принятому порядку

Подтверждение
Проверка доступности и ключевого сценария

Подготовка настроек выпуска

Проверяем	Ожидаемое состояние
Среда	Выбрана рабочая, а не локальная
Подключение	Доступна нужная база
Адрес	Совпадает с опубликованным маршрутом
Защищённый обмен	Сертификат и HTTPS подходят адресу
Сведения о версии	Можно установить действующую редакцию

Обновление структуры данных

Миграция — управляемое изменение схемы базы, согласованное с версией приложения.

01

Пример изменения

Новая версия использует дополнительный столбец

02

Подготовка

Проверить миграцию и чтение существующих записей на копии

03

Выпуск

Выполнить согласованный порядок схемы и кода; проверить восстановление

Возврат после неудачного выпуска

Резервная копия нужна для восстановления данных; возврат программы требует совместимой схемы.

Ситуация	Действие и ограничение
Сбой только в коде	Вернуть прошлую программу, если схема совместима
Изменена схема	Применить заранее проверенный план совместимости
Нужно восстановить копию	Учесть записи, появившиеся после её создания
Передача сопровождению	Назначить ответственного; указать версию и путь проверки

Практика: Готовность выпуска

ЗАДАЧА

Учебный случай: новая версия ожидает дополнительный столбец.

ЧТО ПРОВЕРИТЬ

Выполните три шага: получите проверяемый результат, сверяйте его с условиями и исправляйте расхождения. После каждого шага меняйтесь ролями.

Командный разбор · 3 шага · проверка каждого результата

https://olgakraven.github.io/2026-PIS-lecture/?webinar=3&slide=77&mode=activity

Готовность выпуска

Учебный случай: новая версия ожидает дополнительный столбец. Код уже собран. Миграция на целевой базе ещё не выполнена. Старая версия должна иметь путь восстановления.

Аналитик выделяет правило · Оператор записывает результат · Проверяющий ищет контрпример. Меняйтесь ролями после каждого шага.

1. Найдите несовместимость

Учебный новый код читает поле orders.priority; в целевой схеме поля ещё нет. Аналитик указывает таблицу и недостающее поле.

Таблица: _____

Поле: _____

Правило и проверка: _____

2. Составьте план переключения

Для данного совместимого расширения: A — переключить код; B — применить миграцию; C — проверить резервную копию и восстановление; D — проверить чтение новой схемы. Оператор задаёт порядок.

Порядок: _____

Шаг изменения схемы: _____

Правило и проверка: _____

3. Проверьте остановку выпуска

На шаге D чтение priority завершилось ошибкой. Переключение ещё не началось. Проверяющий указывает последний разрешённый шаг и число выполненных переключений.

Последний шаг: _____

Переключений: _____

Правило и проверка: _____

Прогноз → проверенные результаты → уточнённый вывод. Сверяйте каждый шаг в интерактивном дополнении; обоснование обсудите с преподавателем.

Самопроверка · Вопрос 8

Достаточно ли наличия файла резервной копии?

Ваш ответ

- ○ **А.** Да
- ○ **Б.** Нет, нужно проверить восстановление и совместимость
- ○ **В.** Да, если имя содержит дату

Итоги занятия

Что связывает рассмотренные решения

Связь	Результат
Запрос → полномочие	Сеанс определяет пользователя; тело запроса не доказывает право.
Запись → подтверждение	UNIQUE защищает пару; COMMIT предшествует ответу 201.
Выпуск → проверка	Конфигурация, совместимость схемы и восстановление проверяются отдельно.

Вопросы от аудитории

- Почему номер студента не передаётся при создании?
- Что произойдёт при двух одновременных запросах?
- Когда можно показать подтверждение?

Самопроверка: ответы и пояснения

Сначала выберите ответ самостоятельно, затем сравните обоснование.

Вопрос 1

Браузер отправил id пользователя в JSON. Можно доверять этому id?

Нет, сервер устанавливает личность по сеансу

Формат и скрытое поле не доказывают личность отправителя.

Вопрос 2

JSON прочитан без синтаксической ошибки, но мотивация содержит 49 символов. Что следует?

Формат корректен, но предметное правило длины не выполнено

Синтаксис описывает структуру сообщения; допустимость значения проверяется отдельно. По договору минимум равен 50 после нормализации.

Вопрос 3

GET своих заявок возвращает чужие записи. Что нарушено?

Фильтрация по установленному пользователю

Ресурс «свои заявки» должен ограничивать выборку текущим пользователем.

Вопрос 4

Ирина имеет роль teacher и владеет проектом, но заявка уже accepted. Что проверить?

Допустимость перехода из текущего состояния

Роль и принадлежность дают полномочие на объект, но не разрешают переход, которого нет в модели. Итоговое состояние сохраняется.

Самопроверка: ответы и пояснения

Сначала выберите ответ самостоятельно, затем сравните обоснование.

Вопрос 5

Два запроса одновременно проверили отсутствие пары. Что защищает от двух записей?

Уникальное ограничение в базе и обработка конфликта

Проверка перед записью не устраняет гонку; ограничение проверяется при записи.

Вопрос 6

После восстановления связи GET возвращает заявку 104, а прежний POST завершился таймаутом. Что установлено?

Заявка 104 существует и доступна текущему пользователю

Успешное чтение устанавливает текущее сохранённое состояние. Сам таймаут не определял, был ли выполнен COMMIT.

Вопрос 7

Куда передавать пароль базы при выпуске?

В защищённую конфигурацию сервера

Секрет подключения нужен серверу; клиентский пакет и URL его раскрывают.

Вопрос 8

Достаточно ли наличия файла резервной копии?

Нет, нужно проверить восстановление и совместимость

Пригодность копии подтверждается восстановлением; выпуск также требует совместимости приложения и схемы.